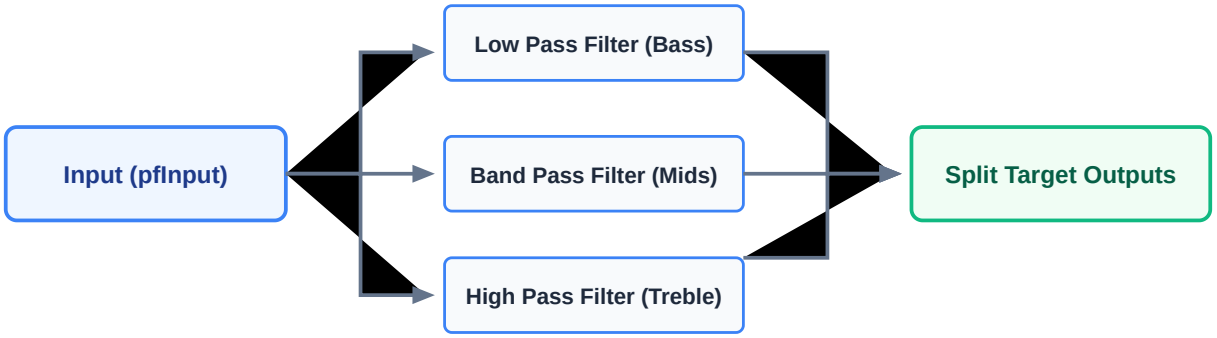
Audio Module Documentation: Equalizer Block

A simple breakdown of how the 3-band equalizer splits and processes our live microphone stream using biquad filter coefficients.

---

1. High-Level Diagram

This diagram tracks the parallel data routing inside the Equalizer Block. The single incoming audio block from the live microphone is copied simultaneously into three separate biquad filters (Low Pass, Band Pass, and High Pass) to break the spectrum down into Bass, Mids, and Treble frequencies.



2. Understanding Filter Coefficients (b0, b1, b2, a0, a1, a2)

Each of the three equalizer bands uses a Second-Order IIR Filter, commonly known as a **Biquad**. The shape, frequency cutoff, and performance of these filters are entirely determined by two sets of mathematical coefficients: the **Feed-forward (b)** values and the **Feedback (a)** values.

Feed-Forward Coefficients (The 'b' Array)

The `b` coefficients act directly on the **incoming, fresh audio samples**. They define what are known as the "zeros" of the system transfer function and govern the immediate tone styling.

- **`b0` (Current Input Scale)**: Controls how much of the brand-new, current audio sample at index `x[n]` is allowed into the filter.
- **`b1` (One-Sample Delayed Input Scale)**: Controls the weight of the previous audio sample `x[n-1]` that arrived exactly one sample clock cycle ago.
- **`b2` (Two-Sample Delayed Input Scale)**: Controls the weight of the historical audio sample `x[n-2]` that arrived two sample clock cycles ago.

Feedback Coefficients (The 'a' Array)

The `a` coefficients act on **past, already processed output data** stored in our history registers. They create the system's "poles", establishing how the filter resonates and sustains frequencies.

- **`a0` (The Normalization Baseline Factor)**: Represents the scaling factor of the current output sample `y[n]`. In standard fixed DSP code implementations, the entire difference equation is divided by `a0`. Because your system handles preset coefficients pre-normalized to 1.0f, `a0` is implicitly treated as 1, which lets us skip an expensive division loop on our ESP32 hardware!
- **`a1` (One-Sample Delayed Output Scale)**: Controls how much of the immediate past filtered sample `y[n-1]` is fed back into the math loop. Notice that in the code execution array, it is subtracted (using a minus sign) to maintain stability.

- **`a2` (Two-Sample Delayed Output Scale)**: Controls the feedback strength of the older filtered sample `y[n-2]` from two cycles ago.

3. Module Structures & Memory Footprint

The equalizer groups independent nested filter configurations to build a unified multi-band module architecture cleanly.

Configuration Structure (`equalizer_config_t`)

This aggregates individual coefficient datasets required by our underlying biquad cascades during startup configuration steps.

- **`low_pass_config_t low`** : Holds the 5 standard filter coefficient values (`{b0, b1, b2, a1, a2}`) for the Bass processing block.
- **`band_pass_config_t mid`** : Holds the 5 standard coefficient values targeting the Mid range block.
- **`high_pass_config_t high`** : Holds the 5 standard coefficient values mapping out the Treble frequency block.

Workspace Handle Structure (`equalizer_hdl_t`)

Houses persistent instance states. It maintains structural memory isolations for each active sub-module channel concurrently.

- **`low_pass_hdl_t low_hdl`** : State variables and history buffers mapping the Bass cascade pipeline.
- **`band_pass_hdl_t mid_hdl`** : State variables tracking the voice and speech frequency range.
- **`high_pass_hdl_t high_hdl`** : State tracking details for high crisp brilliance or air bands.

Static Footprint Analysis

Sub-Module Target Tracker	Internal Data Arrays	Byte Weight Footprint	Dynamic Heap Overhead
`low_hdl` (Bass Sub-unit)	State history mapping variables	Variable (~40 bytes)	0 Bytes (Statically Managed)
`mid_hdl` (Mid Sub-unit)	State history mapping variables	Variable (~40 bytes)	0 Bytes (Statically Managed)
`high_hdl` (Treble Sub-unit)	State history mapping variables	Variable (~40 bytes)	0 Bytes (Statically Managed)

4. Simple Parallel Math Logic

Unlike serial modules, a standard equalizer block uses a parallel topology. It does not overwrite a single stream. Instead, it reads the input stream and feeds it into three mathematically independent channels simultaneously.

The processing architecture loops through frames implementing these three tasks in parallel:

$$\text{Bass\_Output}[n] = \text{Filter\_Low}(\text{Input}[n])$$
$$\text{Mids\_Output}[n] = \text{Filter\_Mid}(\text{Input}[n])$$
$$\text{Treble\_Output}[n] = \text{Filter\_High}(\text{Input}[n])$$

By outputting three isolated frequency streams instead of immediately summing them together inside the loop, the calling routine gains total freedom to change the volume of each band individually using external gain blocks before final mixing occurs.

5. Lifecycle Functions & Loop Execution

The equalizer execution layout follows our standard lifecycle plan. We treat pointer overlaps carefully. Because this module writes out three distinct destination arrays simultaneously, passing shared tracking arrays can lead to memory clobbering. This behavior contract is explicitly detailed below and documented directly in the public header.

A. Query Memory Size (`equalizer_open`)

```
STATUS equalizer_open(uint32_t *pui32Size) {
   if (pui32Size == NULL) return STATUS_NOT_OK;
   *pui32Size = sizeof(equalizer_hdl_t);
   return STATUS_OK;
}
```

What it does: Tells our framework exactly how many bytes are needed to allocate the structural handle instance safely.

B. Initialize Module Settings (`equalizer_init`)

```
STATUS equalizer_init(equalizer_hdl_t *phdl, const equalizer_config_t *psConfig) {
   if (phdl == NULL || psConfig == NULL) return STATUS_NOT_OK;

   low_pass_init(&phdl->low_hdl, &psConfig->low);
   band_pass_init(&phdl->mid_hdl, &psConfig->mid);
   high_pass_init(&phdl->high_hdl, &psConfig->high);

   return STATUS_OK;
}
```

What it does: Links the nested sub-handles and passes the starting frequency coefficient groups down into their respective initialization targets.

C. Process Audio Samples (`equalizer_process`)

```
/**
* @brief Splits an input stream into three isolated frequency bands.
* @note [NON-IN-PLACE COMPATIBLE] This function writes out multiple streams
* simultaneously in parallel. The output target buffers (pfLowOutput, pfMidOutput,
```

```
 * and pfHighOutput) must point to independent regions and cannot share pointers
 * with pfInput or each other.
 */
STATUS equalizer_process(
    equalizer_hdl_t *phdl,
    const float *pfInput,
    float *pfLowOutput,
    float *pfMidOutput,
    float *pfHighOutput,
    uint32_t ui32NumSamples)
{
    if (phdl == NULL || pfInput == NULL || pfLowOutput == NULL || pfMidOutput == NULL ||
pfHighOutput == NULL) {
        return STATUS_NOT_OK;
    }

    // Pass the input stream into all three biquad channels in parallel
    low_pass_process(&phdl->low_hdl, pfInput, pfLowOutput, ui32NumSamples);
    band_pass_process(&phdl->mid_hdl, pfInput, pfMidOutput, ui32NumSamples);
    high_pass_process(&phdl->high_hdl, pfInput, pfHighOutput, ui32NumSamples);

    return STATUS_OK;
}
```

What it does: Feeds the current live microphone audio frame into each filter path. Safety depends strictly on implementation loops. Since this function splits one stream into three separate arrays, it is 'strictly non-in-place'. The output arrays must be completely isolated to avoid corrupting data mid-loop.

D. Close and Clean Up (`equalizer_close`)

```
 STATUS equalizer_close(equalizer_hdl_t *phdl) {
    if (phdl == NULL) return STATUS_NOT_OK;

    low_pass_close(&phdl->low_hdl);
    band_pass_close(&phdl->mid_hdl);
    high_pass_close(&phdl->high_hdl);

    return STATUS_OK;
}
```

What it does: Safely closes out the nested sub-units. Since all workspace state handles are statically allocated across the scope, no dynamic `free()` calls are executed.